

Linux Processes Inside the Kernel

A comprehensive, undergraduate-friendly guide to how Linux creates, runs, schedules, and destroys processes: from `fork()` and `exec()` in your C program to `task_struct`, page tables, signals, pipes, and the scheduler.

Generated for CMSC 125 ICS UPLB

Reviewed by JACH

Includes ten diagrams, annotated kernel walk-throughs, six hands-on labs with real output, and review questions with answers.

Contents

Before You Begin	5
The whole story in one paragraph	5
Prerequisites	5
Roadmap	5
What Is a Process?	7
1.1 Program vs process	7
1.2 What a process consists of	7
1.3 Why the OS needs processes	7
1.4 Two modes, one system call boundary	8
1.5 Processes and threads on Linux	8
The Process Address Space	9
2.1 The segments	9
2.2 Seeing it yourself: /proc/PID/maps	10
2.3 Virtual memory in the kernel	10
2.4 ASLR: why addresses change every run	11
The Kernel's View: task_struct	12
3.1 The process descriptor	12
3.2 How the kernel finds processes	13
3.3 Process identity	13
3.4 The special processes	13
Process States and Lifecycle	14
4.1 Watching the states change	14
Creating Processes: fork()	16
5.1 What fork() does	16
5.2 What the child gets	16
5.3 Copy-on-write: why fork() is fast	16
5.4 Inside the kernel	17
5.5 vfork(), posix_spawn(), and clone() flags	17
5.6 Things to know	18
Running Other Programs: exec()	19
6.1 What exec() does	19
6.2 What survives an exec	19

6.3 The fork-exec pattern: how a shell runs a command	19
6.4 Inside execve()	20
Termination, wait(), Zombies, and Orphans	22
7.1 How processes end	22
7.2 What the kernel does at exit	22
7.3 wait(): reaping children	22
7.4 Zombies	23
7.5 Orphans and reparenting	23
7.6 The classic daemon recipe	24
Scheduling and Context Switching	25
8.1 Run queues	25
8.2 Scheduling classes and policies	25
8.3 The fair scheduler: CFS and EEVDF	25
8.4 The context switch	26
Signals	27
9.1 Where signals come from	27
9.2 Common signals	27
9.3 How a signal is delivered	28
9.4 Handling signals in C	28
Inter-Process Communication (IPC)	29
10.1 Pipes in detail	29
10.2 Shared memory in one page	30
Process Groups, Sessions, and Job Control	31
11.1 How job control works	31
Limits, Namespaces, and cgroups	33
12.1 Resource limits (rlimits)	33
12.2 Namespaces: giving processes their own view	33
12.3 cgroups: limiting what processes may use	34
Observing Processes	35
13.1 Everyday commands	35
13.2 The /proc filesystem	35
Hands-On Labs	36
Lab 1: fork() and private copies	36
Lab 2: fork + exec + wait, and exit codes	36

Lab 3: Zombies and orphans	37
Lab 4: Signals between processes	38
Lab 5: Pipes and a mini pipeline	39
Lab 6: Explore a live process through /proc	40
Review, Misconceptions, and Reference	42
15.1 Common misconceptions	42
15.2 Where to find it in the source	42
15.3 Review questions	43
15.4 Answer key	43
15.5 Glossary	44
15.6 References and further reading	45

START HERE

Before You Begin

Every command you type, every app you open, and every service running on a server is a **process**. The process is the most important abstraction in an operating system: it is how the OS turns one computer into many seemingly independent ones. This guide explains what a Linux process really is, how the kernel represents it, and how processes are created, run, communicate, and die.

As in an OS course, we keep one question in mind: *what does the program ask for, and what does the kernel do about it?* Whenever you see a function like `fork()` or `wait()`, we will follow it into the kernel.

The whole story in one paragraph

A **program** is a file on disk. A **process** is a running instance of a program: it has its own virtual address space, CPU state, open files, and identity. The kernel tracks each process in a structure called `task_struct`. New processes are made with `fork()`, which clones the caller cheaply using copy-on-write memory. The clone then usually calls `exec()` to replace itself with a different program. When a process ends, it becomes a **zombie** holding only its exit status until its parent collects it with `wait()`. In between, the kernel **scheduler** shares the CPUs among all runnable processes, **signals** deliver asynchronous events, and **pipes** and other IPC mechanisms let processes cooperate. Every shell command you have ever run used exactly this recipe.

Prerequisites

- Basic C: pointers, structs, `printf`, and reading man pages.
- OS lecture basics: user mode vs kernel mode, system calls, interrupts, virtual memory.
- A Linux shell. The labs use `gcc`, `ps`, `kill`, and `strace`.

Scope. This guide is self-contained and focuses on whole processes. Threads appear only where they touch the kernel's design (for example, in `clone()` and `task_struct`), and are explained briefly where they do.

Roadmap

Chapter	Question it answers
1 What is a process?	Program vs process; what belongs to a process; why the OS needs them
2 The address space	What do the stack, heap, and code look like? What is <code>/proc/PID/maps</code> ?
3 The kernel's view	What is <code>task_struct</code> ? What are PID, PPID, PGID, SID? Who is PID 1?
4 States and lifecycle	What do R, S, D, T, Z mean in <code>ps</code> ?
5 Creating: <code>fork()</code>	How can Linux clone a process so quickly? What is copy-on-write?
6 Running programs: <code>exec()</code>	How does a shell run <code>ls</code> ? What happens inside <code>execve</code> ?
7 Termination and <code>wait()</code>	Zombies, orphans, exit codes, and who cleans up
8 Scheduling	How does the kernel decide who runs next? What is a context switch?
9 Signals	How do Ctrl-C, <code>kill</code> , and segfaults reach a process?
10 Inter-process communication	Pipes, shared memory, sockets: how processes talk

Chapter	Question it answers
11 Groups, sessions, job control	Foreground, background, terminals, daemons
12 Limits, namespaces, cgroups	How Linux restricts processes and builds containers
13 Observing processes	ps , top , /proc , strace : the toolbox
14 Hands-on labs	Six experiments with real output
15 Review	Misconceptions, source map, questions, answers, glossary, references

NOTE — ABOUT THE EXAMPLES

Sample outputs were produced on Ubuntu 24.04 (glibc 2.39) with a Linux 6.x kernel. PIDs, addresses, and timings will differ on your machine, but the *patterns* will match. Kernel code shown is **simplified** for learning; real functions have many extra checks.

CHAPTER 1

What Is a Process?

1.1 Program vs process

A **program** is passive: an executable file (on Linux, usually in **ELF** format) sitting on disk, containing machine code and data. A **process** is active: a program that has been loaded and is running (or is ready or waiting to run). The same program can be several processes at once. If you open three terminal windows, you have three separate processes running the same **bash** program, each with its own variables and current directory.

KEY IDEA

Analogy: a program is a **recipe** in a cookbook. A process is a **cook actively following that recipe** in a kitchen, with their own ingredients, their own place in the steps, and their own half-finished dish. Three cooks can follow the same recipe at the same time without mixing up their bowls.

1.2 What a process consists of

Component	What it is	Where it lives
Address space	Code, global data, heap, stack, mapped files, all in virtual memory	Described by <code>mm_struct</code>
CPU context	Program counter, stack pointer, general registers, flags	Saved in the task's kernel stack when not running
Open files	File descriptor table (0 = stdin, 1 = stdout, 2 = stderr, ...)	<code>files_struct</code>
Environment	Current directory, root, umask, environment variables, argv	<code>fs_struct</code> , user memory
Signal state	Handlers, blocked-signal mask, pending signals	<code>sighand_struct</code> , <code>signal_struct</code>
Identity	PID, parent PID, process group, session, UID/GID	<code>task_struct</code> , <code>cred</code>
Scheduling state	State (running, sleeping, ...), priority, CPU time used	<code>task_struct</code> , <code>sched_entity</code>
Accounting	CPU time, memory use, page faults, context switch counts	<code>task_struct</code>

1.3 Why the OS needs processes

- **Multiprogramming.** Many programs share few CPUs. The kernel switches among processes so quickly that each appears to have its own CPU.
- **Isolation and protection.** Each process has a private virtual address space. A buggy program cannot overwrite another program's memory or the kernel's. When it crashes, only it dies.
- **Resource accounting and control.** Because resources are tied to processes, the kernel can limit, measure, and bill them: CPU time, memory, open files, and more.

- **A unit of ownership.** Permissions are checked against the user ID of a process. Killing a process is a way to reclaim everything it holds.

1.4 Two modes, one system call boundary

The CPU runs a process in **user mode** (restricted: no direct hardware access) or **kernel mode** (full privileges). A process enters the kernel in three ways: a **system call** (a deliberate request such as `fork` or `read`), an **exception** (like a page fault or divide by zero), or an **interrupt** (a timer tick or device event). The kernel runs on behalf of the current process, does the work, then returns to user mode, possibly to a *different* process if the scheduler decides so.

1.5 Processes and threads on Linux

Linux does not have separate kernel objects for processes and threads. Both are **tasks**, described by `task_struct`. A traditional single-threaded process is one task. A multithreaded process is several tasks that share an address space, file table, and signal handlers; each of those tasks is what programmers call a **thread**. In this guide, "process" usually means a task that shares nothing with its parent after creation.

CHAPTER 2

The Process Address Space

Each process sees a private, contiguous range of **virtual addresses**. The hardware's **MMU** (memory management unit) and the kernel's **page tables** translate those virtual addresses to physical RAM in 4 KiB **pages**. Two processes can use the same virtual address for completely different data, because each has its own page tables.

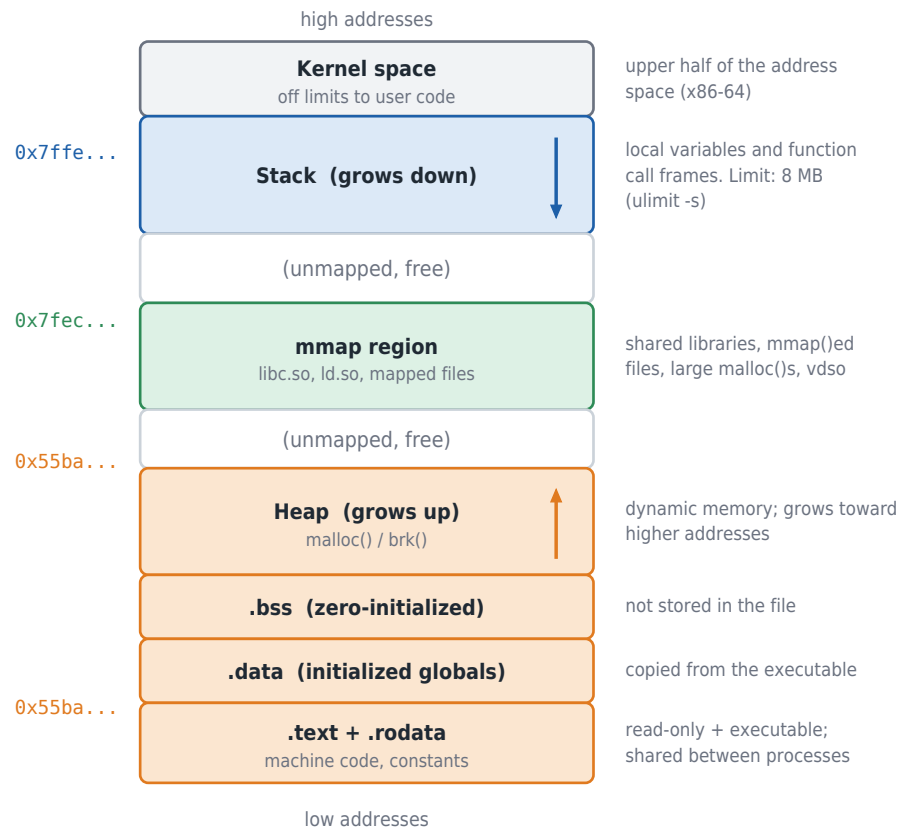


Figure 2.1: Typical layout of a 64-bit Linux process. The addresses on the left come from Lab 6.

2.1 The segments

Region	Contents	Notes
Text (.text)	Machine instructions	Read + execute, not writable. Backed by the executable file; one copy in RAM can be shared by all processes running that program.
.rodata	String literals, constants	Read-only.
.data	Global/static variables with non-zero initial values	Copied from the file; writable and private.
.bss	Global/static variables initialized to zero	Takes no space in the file; the kernel supplies zero pages on demand.

Region	Contents	Notes
Heap	Memory from <code>malloc</code> , <code>calloc</code>	Small requests grow the heap with <code>brk()</code> . Large requests (about 128 KiB and up by default) use <code>mmap()</code> .
mmap region	Shared libraries (libc.so), memory-mapped files, big allocations, thread stacks	Between heap and stack. The dynamic linker <code>ld.so</code> maps libraries here at startup.
Stack	Local variables, arguments, return addresses	Grows downward, automatically, up to <code>ulimit -s</code> (usually 8 MB). One per thread.
vdso / vvar	Kernel-provided code and data mapped into user space	Lets calls like <code>clock_gettime()</code> run without entering the kernel.
Kernel space	The kernel itself	In the upper part of the address space. User code faults if it touches it.

2.2 Seeing it yourself: /proc/PID/maps

Every process has a file `/proc/<PID>/maps` listing its memory regions. Each line is one contiguous region called a **VMA** (virtual memory area). Here is a trimmed version from Lab 6:

```

address range      perms mapped from
55ba429fb000-55ba429fc000 r--p /home/claude/plabs/p6
55ba429fc000-55ba429fd000 r-xp /home/claude/plabs/p6    <- .text (the code)
55ba429fd000-55ba429fe000 r--p /home/claude/plabs/p6    <- .rodata
55ba429ff000-55ba42a00000 rw-p /home/claude/plabs/p6    <- .data + .bss
55ba743dd000-55ba743fe000 rw-p [heap]
7fec07228000-7fec073b0000 r-xp /usr/lib/x86_64-linux-gnu/libc.so.6    <- libc code
7fec07403000-7fec07405000 rw-p /usr/lib/x86_64-linux-gnu/libc.so.6    <- libc data
7fec075ec000-7fec075f0000 r--p [vvar]
7fec075f2000-7fec075f4000 r-xp [vdso]
7fec075f5000-7fec07620000 r-xp /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe11cd000-7ffe11ee000 rw-p [stack]
```

- The permission letters are **r**ead, **w**rite, **e**xecute, and **p**rivate (copy-on-write) or **s**hared. Code is `r-x`, never writable; data is `rw-`, never executable. This blocks a large class of attacks.
- Even a tiny program maps libc and the dynamic linker, since almost every C program uses them.
- The stack region here is 132 KiB, not 8 MB: it **grows on demand** when the process touches lower addresses.

2.3 Virtual memory in the kernel

Inside the kernel, a process's address space is described by an `mm_struct`. It holds a collection of `vm_area_struct` objects (one per line of `maps`; a maple tree in recent kernels), a pointer to the top-level **page table**, and counters such as total and resident memory. Nothing is loaded eagerly: pages are brought in by **demand paging**.

- 1 The program touches a virtual address that has no page-table entry yet (or one marked not-present).
- 2 The CPU raises a **page fault**, entering the kernel.
- 3 The kernel finds the VMA covering that address. If none does, the process gets `SIGSEGV`: a segmentation fault.

- 4 If it does, the kernel allocates a page frame and fills it (zeros for the heap or bss, file contents for code and mapped files), updates the page table, and returns.
- 5 The instruction is retried and now succeeds. The program never noticed.

KEY IDEA

Virtual size is not real memory. In Lab 6, `VmSize` (everything mapped) was about 2.7 MB while `VmRSS` (pages actually in RAM) was 1.7 MB. A process can map far more than it uses; the kernel only spends RAM on pages that are touched.

2.4 ASLR: why addresses change every run

Run the same program twice and print an address on the stack or heap. You will get different values each time. This is **Address Space Layout Randomization**: the kernel randomizes the base of the stack, heap, mmap region, and (for position-independent executables) the program itself, so an attacker cannot predict where code or data lives. Real output from Lab 6:

```
heap (malloc)      : 0x558f58eb52a0      <- run 1
stack (local)      : 0x7fff0d98869c
heap (malloc)      : 0x5644834942a0      <- run 2: completely different
stack (local)      : 0x7ffc90c916cc
```

The setting is in `/proc/sys/kernel/randomize_va_space` (2 = full randomization, the usual default). Debuggers switch it off for repeatable runs.

CHAPTER 3

The Kernel's View: task_struct

3.1 The process descriptor

For every process, the kernel keeps a `struct task_struct` (defined in `include/linux/sched.h`). Textbooks call this the **Process Control Block (PCB)**. It is a large structure: it stores the process's state, identity, and *pointers* to other structures that describe its resources.

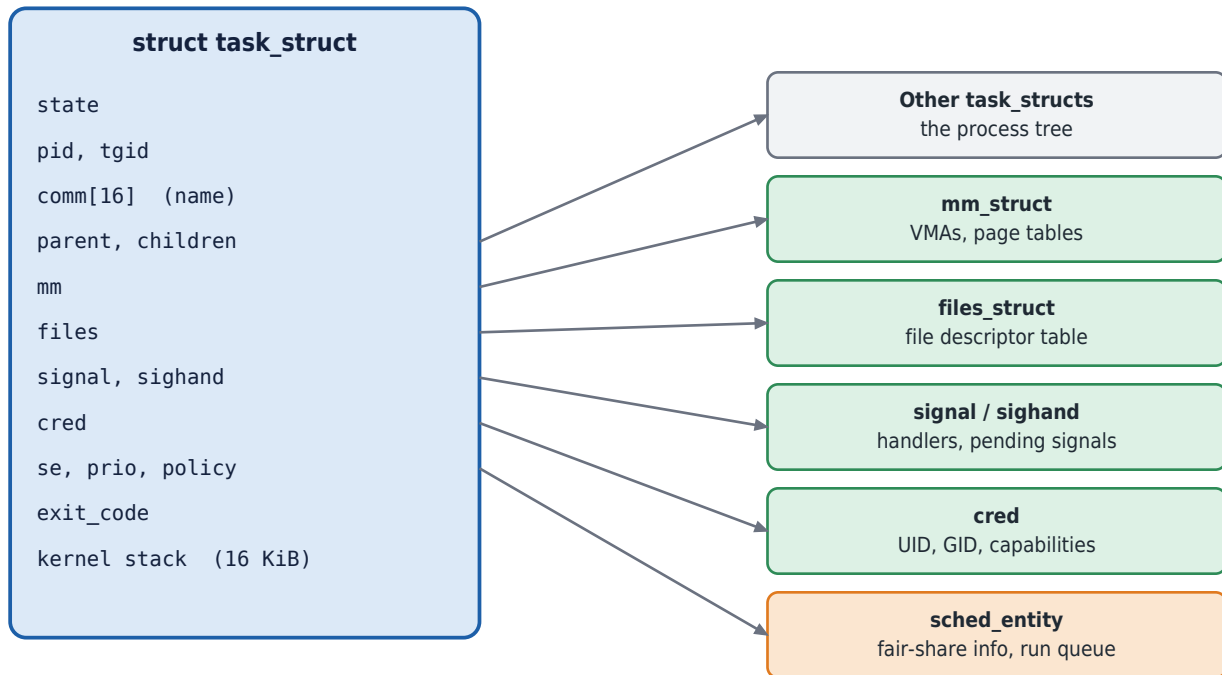


Figure 3.1: A simplified `task_struct`. Shared resources are separate objects, which is why `clone()` can share or copy each one independently.

Field	Meaning
<code>state</code>	Running/runnable, sleeping, stopped, zombie, ... (Chapter 4)
<code>pid, tgid</code>	Task ID and thread-group ID. <code>getpid()</code> returns <code>tgid</code> ; for a single-threaded process the two are equal
<code>comm[16]</code>	Short command name (what <code>ps</code> and <code>top</code> display); 15 characters plus NUL
<code>real_parent, parent, children, sibling</code>	Links forming the process tree
<code>mm</code>	The address space (<code>mm_struct</code>). NULL for kernel threads , which have no user memory
<code>files, fs</code>	Open-file table; current directory and root
<code>signal, sighand</code>	Signal handlers and pending signals
<code>cred</code>	User and group IDs, capabilities (who is this process allowed to act as?)
<code>se, prio, policy</code>	Scheduler data: fair-share bookkeeping, priority, scheduling policy

Field	Meaning
<code>exit_code</code>	Exit status kept for the parent after the process dies
kernel stack	16 KiB per task on x86-64; used while the task is inside a system call or interrupt

3.2 How the kernel finds processes

- All tasks are linked in a circular doubly linked list starting at `init_task`. Kernel code can walk it with the macro `for_each_process()`. This is what `ps` effectively shows you through `/proc`.
- PIDs are allocated by an ID allocator (`kernel/pid.c`). The maximum is in `/proc/sys/kernel/pid_max` (32768 on this machine; many distributions raise it to 4194304). When the numbers run out, allocation wraps around and skips PIDs still in use.
- Each task has a kernel-internal pointer to its parent and a list of children, so **the process tree is a real data structure**, not just a display.

3.3 Process identity

ID	Name	Meaning	Get it with
PID	Process ID	Unique number for this process	<code>getpid()</code>
PPID	Parent PID	Who created me (and who will <code>wait()</code> for me)	<code>getppid()</code>
PGID	Process group ID	Set of related processes, e.g. one shell pipeline (Chapter 11)	<code>getpgid()</code>
SID	Session ID	Group of process groups tied to one login/terminal	<code>getsid()</code>
UID / EUID	Real / effective user ID	Who owns the process / whose permissions are used for checks	<code>getuid()</code> , <code>geteuid()</code>
GID / EGID	Group IDs	Same idea for groups	<code>getgid()</code> , <code>getegid()</code>

3.4 The special processes

- PID 0: the idle task** (`swapper`). One per CPU; runs when nothing else is runnable. Not shown by `ps`.
- PID 1: init** (usually `systemd`). The first user-space process, started by the kernel. It is the ancestor of every user process, and it adopts orphans (Chapter 7).
- PID 2: kthreadd**. The parent of all **kernel threads**: tasks that run kernel code only and have no user address space. `ps` shows them in square brackets.

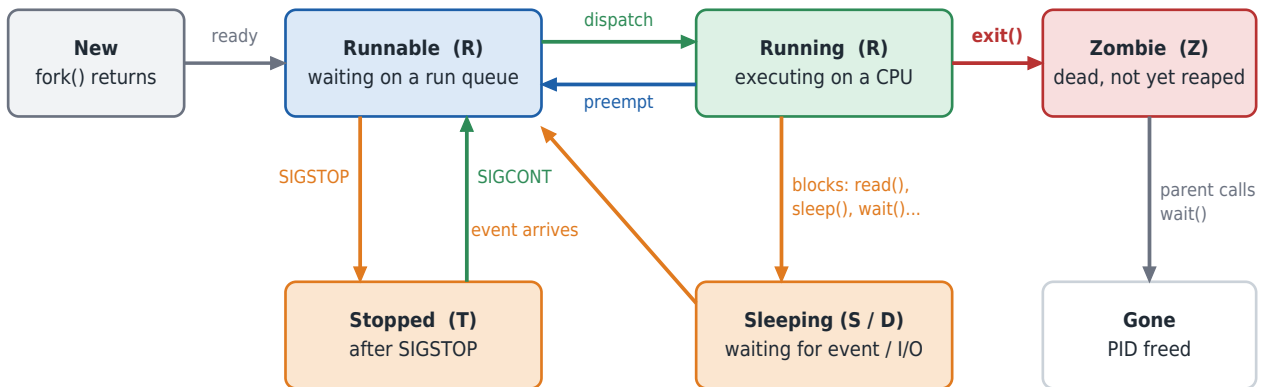
The kernel starts PID 1 and PID 2 during boot; everything else descends from PID 1 through `fork()`. Real `ps` output (kernel threads are children of PID 2):

```
$ ps -eo pid,ppid,pgid,sid,tt,stat,cmd
PID  PPID  PGID   SID  TT      STAT  CMD
  2    0    0     0  ?       S      [kthreadd]
  3    2    0     0  ?       S      [pool_workqueue_release]
  4    2    0     0  ?       I<     [kworker/R-rcu_gp]
  5    2    0     0  ?       I<     [kworker/R-sync_wq]
```

CHAPTER 4

Process States and Lifecycle

Textbooks often draw a five-state model: new, ready, running, waiting, terminated. Linux has the same ideas but slightly different names and finer detail.



Linux reports "runnable" and "running" with the same letter, R. It also has D (uninterruptible sleep) and X (dead). A zombie keeps only its PID and exit status until the parent reaps it.

Figure 4.1: Process states and transitions on Linux, with the letters shown by `ps`.

ps letter	Kernel name	Meaning	Typical cause
R	<code>TASK_RUNNING</code>	Running on a CPU or waiting on a run queue for one	CPU-bound work; just woken up
S	<code>TASK_INTERRUPTIBLE</code>	Sleeping until an event; can be woken by a signal	<code>sleep()</code> , <code>read()</code> on a pipe or socket, <code>wait()</code>
D	<code>TASK_UNINTERRUPTIBLE</code>	Sleeping and ignores signals (even <code>SIGKILL</code>) until the operation completes	Waiting for disk or network filesystem I/O
T	<code>__TASK_STOPPED</code>	Stopped by a job-control signal	Ctrl-Z (<code>SIGTSTP</code>), <code>SIGSTOP</code>
t	<code>__TASK_TRACED</code>	Stopped by a debugger	<code>gdb</code> , <code>strace</code> breakpoints
Z	<code>EXIT_ZOMBIE</code>	Finished; waiting for the parent to read its exit status	Parent has not called <code>wait()</code> yet
X	<code>EXIT_DEAD</code>	Being removed; you almost never see it	Final moments
I	idle	Idle kernel thread (shown for kernel threads only)	Kernel worker with nothing to do

`ps` may add extra characters after the state letter: `s` (session leader), `+` (in the foreground process group), `l` (multithreaded), `<` (high priority), `N` (low priority).

4.1 Watching the states change

This is real output from Lab 6. A **sleep** process is created, then stopped and continued with signals. The **WCHAN** column shows the kernel function where the process is sleeping:

```
$ sleep 100 &                                # then, repeatedly: ps -o pid,stat,wchan:14,cmd -p 181
181 S      hrtimer_nanosl sleep 100           <- S: waiting for a timer (WCHAN names the kernel function)
$ kill -STOP 181
181 T      do_signal_stop sleep 100           <- T: stopped
$ kill -CONT 181
181 S      do_sys_restart sleep 100           <- back to sleeping

$ ./spin &                                    # an infinite loop: for(;;);
197 R      ./spin                             <- R: always runnable or running
```

WATCH OUT

A process in state **D** cannot be killed, not even with **kill -9**, because it is in the middle of an operation the kernel must finish. If a process is stuck in D for a long time, the usual culprit is a hung disk or network mount. Linux counts both **R** and **D** tasks in the "load average" shown by **uptime**, which is why a slow disk can push the load up even when the CPU is idle.

CHAPTER 5

Creating Processes: fork()

5.1 What fork() does

`fork()` creates a new process that is a near-exact **copy** of the caller. It is called once but **returns twice**: once in the parent (returning the child's PID) and once in the child (returning 0). That difference is how the two copies tell themselves apart.

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork");           /* failed: no child was created */
} else if (pid == 0) {
    /* CHILD: runs only here */
} else {
    /* PARENT: pid holds the child's PID */
}
```

5.2 What the child gets

Copied or shared with the child	Different in the child
A copy of the address space (code, data, heap, stack), using copy-on-write	A new, unique PID
A copy of the file descriptor table; the open files are shared (see below)	PPID is the parent's PID
Same current directory, umask, environment variables	Pending signals and timers are cleared
Same signal handlers and signal mask	Resource-usage counters (CPU time) start from zero
Same UID/GID, process group, and session	Record locks (<code>fcntl</code>) and memory locks are not inherited
Same program counter: both continue right after <code>fork()</code>	Only the calling thread exists in the child

WATCH OUT

The child's file descriptors point to the **same open file description** as the parent's. They share the file offset. If both write to the same file, their output interleaves rather than overwrites. This is exactly what makes `cmd1 > out.txt` inherited by children behave sensibly.

5.3 Copy-on-write: why fork() is fast

Copying the whole address space of a large process would be slow and wasteful, especially since the child usually calls `exec()` a moment later and throws the copy away. So Linux uses **copy-on-write (COW)**:

- 1 At `fork()`, the kernel copies the **page tables**, not the pages themselves. Parent and child now point to the same physical pages.
- 2 Every writable private page is marked **read-only** in both processes and its reference count is increased.
- 3 When either process *writes* to such a page, the CPU raises a page fault. The kernel sees the COW situation, allocates a new page, copies the 4 KiB, maps it writable for the writer, and lets the write proceed.

- 4 Pages that are never written are never copied, and the code segment is shared for good.

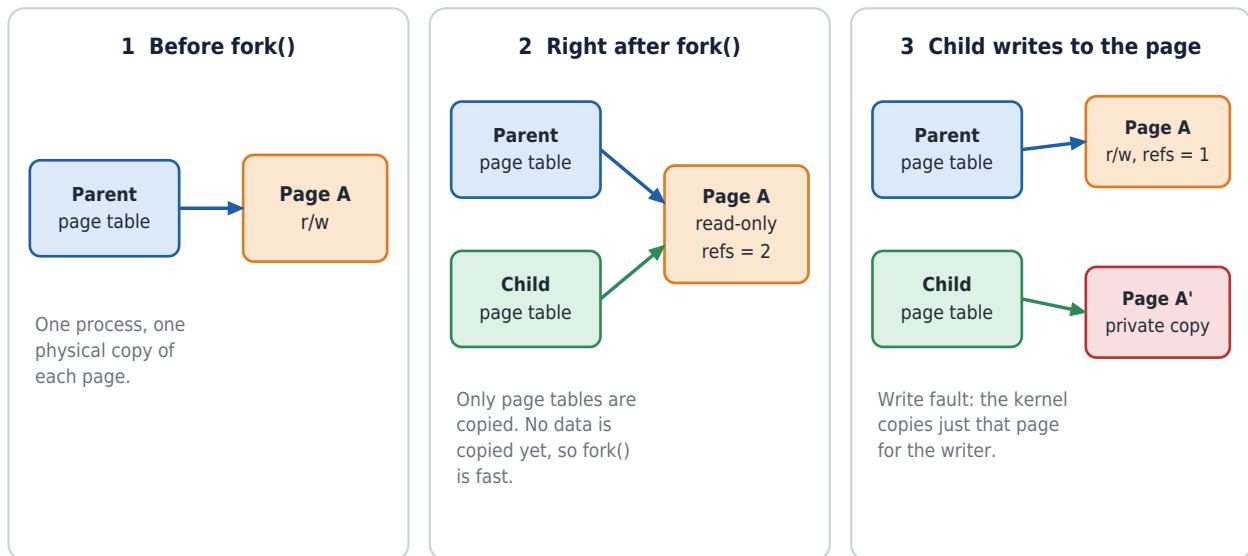


Figure 5.1: Copy-on-write. The cost of fork() depends on the size of the page tables, not on the amount of memory in use.

5.4 Inside the kernel

On Linux, `fork()` is just a special case of the general `clone()` system call (the same general system call that creates threads). glibc's `fork()` invokes `clone` with the flag `SIGCHLD` and *no sharing* flags, so the kernel duplicates everything. Simplified:

```
fork()                                // user call, glibc wrapper
-> clone(flags = SIGCHLD)              // syscall: no CLONE_VM, CLONE_FILES, ...
   -> kernel_clone()                  // kernel/fork.c
       -> copy_process()
           p = dup_task_struct(current); // new task_struct + kernel stack
           copy_creds(), copy_files(), copy_fs(), copy_sighand(), ... // duplicate tables
           copy_mm(): dup_mmap()         // copy VMAs + page tables, mark private pages COW
           p->pid = alloc_pid();          // new PID
           p->real_parent = current;      // link into the process tree
           sched_fork(p);                // initialize scheduler state
           -> wake_up_new_task(p);       // put child on a run queue
           return p->pid (to the parent)  // the child returns 0 when it first runs
```

You can see this in real system-call traces. When the shell in Chapter 6 starts a command, `strace` shows:

```
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD) = 89
```

Creating a *thread* would add flags such as `CLONE_VM` (share the address space), `CLONE_FILES` (share the descriptor table), `CLONE_SIGHAND` (share signal handlers), and `CLONE_THREAD` (join the caller's thread group). None of them appear here, so everything is copied. `SIGCHLD` is the signal that will be sent to the parent when the child exits (Chapter 7).

5.5 vfork(), posix_spawn(), and clone() flags

Call	How it relates to clone()	Use
<code>fork()</code>	<code>clone(SIGCHLD)</code> : copy everything (COW)	General purpose

Call	How it relates to clone()	Use
<code>vfork()</code>	<code>CLONE_VM</code> <code>CLONE_VFORK</code> <code>SIGCHLD</code> : share the address space, and suspend the parent until the child execs or exits	Historic optimization; dangerous, since the child runs on the parent's memory
<code>posix_spawn()</code>	glibc builds it on <code>CLONE_VM</code> <code>CLONE_VFORK</code>	Portable, efficient fork+exec in one call; good for big processes
<code>pthread_create()</code>	clone with <code>CLONE_VM</code> <code>CLONE_FILES</code> <code>CLONE_THREAD</code> ...	Threads

5.6 Things to know

- **Order is not guaranteed.** After `fork()`, the scheduler may run the parent or the child first. Never rely on an order; synchronize with `wait()` or other IPC.
- **`fork()` can fail.** Typical errors are `EAGAIN` (per-user process limit `RLIMIT_NPROC`, system-wide `threads-max`, PID space, or a cgroup `pids.max`) and `ENOMEM`. Always check the return value.
- **Fork bombs.** A program that forks in an unbounded loop can exhaust the process table and freeze a machine. Never test this on a shared server such as a laboratory machine. Limits like `ulimit -u` exist to contain it.
- **Buffered output is duplicated.** The next example shows why.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    printf("hello ");          /* no newline, no fflush: text stays in the stdio buffer */
    fork();                   /* buffer is copied into the child */
    printf("world\n");         /* both processes flush "hello world\n" */
    wait(NULL);
    return 0;
}
```

```
hello world
hello world
```

The word "hello " was still sitting in a user-space buffer when `fork()` copied the process, so both processes printed it. The fix is `fflush(stdout)` before `fork()`.

CHAPTER 6

Running Other Programs: exec()

6.1 What exec() does

`fork()` gives you another copy of *the same* program. To run a *different* program you call one of the **exec** functions. **exec** does not create a process. It **replaces the current process's program**: the old code, data, heap, and stack are discarded and a new program is loaded into the same process. The PID stays the same. On success, **exec never returns**, because the code that called it no longer exists.

Function	Letters decoded
<code>execl(path, arg0, arg1, ..., NULL)</code>	l ist: arguments given one by one
<code>execv(path, argv)</code>	v ector: arguments in an array
<code>execlp, execvp</code>	p : search the directories in PATH for the program name
<code>execle, execve</code>	e : also pass an explicit environment array (execve is the real system call; the others are library wrappers)

6.2 What survives an exec

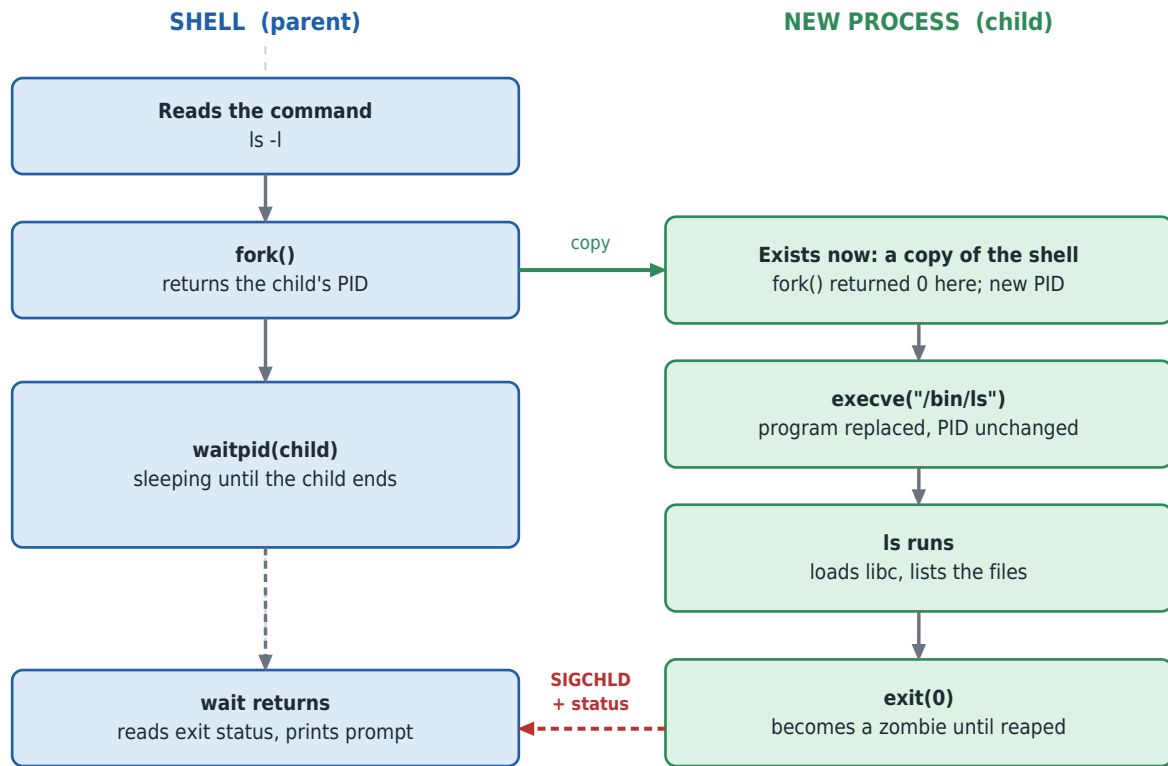
Kept	Replaced or reset
PID, PPID, process group, session	Code, data, heap, stack, all memory mappings
Open file descriptors (unless marked close-on-exec)	Signal handlers reset to default (function pointers no longer exist)
Current directory, umask, UID/GID (unless set-uid)	Other threads: all destroyed
Signal mask; signals set to ignored stay ignored	Environment: whatever the caller passes in envp
Pending signals	Timers, shared-memory attachments, and memory locks

TRY IT

Marking a descriptor **O_CLOEXEC** (or **FD_CLOEXEC**) makes the kernel close it automatically at `exec`. Use it for descriptors the new program should not inherit: it prevents leaks of sockets and file handles into child programs.

6.3 The fork-exec pattern: how a shell runs a command

Why are process creation and program loading two separate steps? Because the gap between them gives the child a chance to prepare, using the parent's full ability to run code, before the new program starts. Redirection (`ls > out.txt`), pipes, and job control all happen in that gap.



Between `fork()` and `execve()`, the child can set up redirections and pipes.

Figure 6.1: What a shell does for every command you type.

Let us watch a real shell run `ls /etc | wc -l`. This `strace` output (trimmed, in original order) is from Lab 6:

```

88 execve("/usr/bin/sh", ["sh", "-c", "ls /etc | wc -l"], ...) = 0
88 pipe2([3, 4], 0) = 0 <- shell makes the pipe
88 clone(child_stack=NULL, flags=...|SIGCHLD ... = 89 <- fork #1
89 dup2(4, 1) = 1 <- child 89: stdout -> pipe
89 execve("/usr/bin/ls", ["ls", "/etc"], ...) <- becomes ls
88 clone(child_stack=NULL, flags=...|SIGCHLD ... = 90 <- fork #2
90 dup2(3, 0) = 0 <- child 90: stdin <- pipe
90 execve("/usr/bin/wc", ["wc", "-l"], ...) <- becomes wc
88 wait4(-1, ... <- shell waits
89 exit_group(0 ...
90 exit_group(0) = ?
88 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 90
88 --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=90, ...} ---
  
```

This one screen contains almost every idea in the guide: `fork` (clone), `exec`, a pipe, `dup2`, `wait4`, `exit`, and a `SIGCHLD` signal. The processes run concurrently, so the trace lines from PIDs 88, 89, and 90 interleave.

6.4 Inside `execve()`

The kernel path (`fs/exec.c`, simplified):

```

execve(path, argv, envp)
-> open the file, read its first bytes (the "magic number")
-> search_binary_handler(): // which format is this file?
    ELF file -> load_elf_binary() // fs/binfmt_elf.c
    "#!" script -> load_script() // re-run with the interpreter, e.g. /bin/bash
  
```

```

    others          -> binfmt_misc, etc.
-> load_elf_binary():
    begin_new_exec()    // point of no return: destroy the old address space and handlers
    map each PT_LOAD segment (code, data) into the fresh address space with mmap
    if the file names an interpreter (PT_INTERP, e.g. ld-linux-x86-64.so.2): map it too
    build the new stack: argc, argv[], envp[], and the "auxiliary vector"
    set the instruction pointer to the entry point (of ld.so, or of the program if static)
-> return to user mode: the process now runs a brand-new program

```

- **Point of no return.** If `execve` fails *before* `begin_new_exec` (file not found, no permission, bad format), it returns `-1` and the old program continues, which is why `perror()` after `exec` makes sense. After that point, failure kills the process.
- **The dynamic linker runs first.** For dynamically linked programs, the first user-space code executed is `ld.so`. It maps `libc` and other libraries, resolves symbols, and only then jumps to `main` via `libc`'s startup code.
- **Shell scripts** work because of the `#!` line: the kernel re-executes the named interpreter with the script as an argument.

CHAPTER 7

Termination, wait(), Zombies, and Orphans

7.1 How processes end

- `return` from `main()`, or calling `exit(status)`: the normal way.
- `_exit(status)`: exits immediately, without flushing stdio buffers or running `atexit` handlers.
- Being **killed by a signal**: for example `SIGSEGV` (segfault), `SIGKILL`, `SIGTERM`, or `abort()` (`SIGABRT`).
- Being killed by the kernel: the **OOM killer** picks a victim when memory runs out.

`exit()` first flushes stdio buffers and runs functions registered with `atexit()`, then calls `_exit()`, which makes the `exit_group` system call and ends the whole process. The **exit status** is a value from 0 to 255 (only the low 8 bits are kept). By convention, 0 means success.

WATCH OUT

In a child after `fork()`, prefer `_exit()` when the child is going to call `exec()` or fail. Plain `exit()` would flush the *copy* of the parent's buffered output a second time (the double "hello world" bug). But if the child has produced output of its own and does *not* `exec`, use `exit()` or `fflush()` first, or that output is lost. Lab 5 shows both.

7.2 What the kernel does at exit

```
do_exit(code)                                // kernel/exit.c
    exit_mm()                                // release the address space (free pages when the last user goes)
    exit_files(), exit_fs()                  // close all open files, drop cwd
    exit_signals(), exit_task_namespaces() ...
    exit_notify():
        reparent any children to init (or the nearest subreaper)
        send SIGCHLD to the parent
        state = EXIT_ZOMBIE                  // task_struct and PID kept, exit status stored
    schedule()                               // never returns: the task is never run again
```

At this point the process is **dead but not gone**. Its memory and files have been released, but its `task_struct` and PID remain, holding the exit status. Why? Because the parent may want to know how it ended, and the kernel cannot hand out the PID again until the parent has had its chance.

7.3 wait(): reaping children

A parent collects a child's exit status with `wait()` or `waitpid()`. This does two things: it **blocks** until a child has exited (if none has yet), and it **reaps** the zombie, letting the kernel free the `task_struct` and the PID (`release_task()`).

```
int status;
pid_t done = waitpid(pid, &status, 0);      /* pid = -1 means "any child" */

if (WIFEXITED(status)) printf("exited with code %d\n", WEXITSTATUS(status));
else if (WIFSIGNALED(status)) printf("killed by signal %d\n", WTERMSIG(status));
```

- **WNOHANG** makes `waitpid` return immediately if no child has finished yet (polling).
- **WIFEXITED**, **WEXITSTATUS**, **WIFSIGNALED**, **WTERMSIG**, and **WIFSTOPPED** decode the packed status integer.
- In the shell, `$?` holds the last exit code. A process killed by signal `N` appears as **128 + N**: for example **139** for **SIGSEGV** (128 + 11). Real check from Lab 6: `sh -c 'kill -SEGV $$'` printed "Segmentation fault" and `$?` was 139.
- Conventional codes: 0 success, 1 general error, 126 found but not executable, 127 command not found.

7.4 Zombies

A **zombie** is a process that has exited but whose parent has not yet called `wait()`. It appears as **Z** (or **<defunct>**) in `ps`. It uses no CPU and no memory except a small kernel structure and, importantly, **a PID**. You cannot kill a zombie with a signal, because it is already dead. The only cure is for the parent to reap it, or for the parent to die (then `init` adopts and reaps it). Real output from Lab 3:

```
$ ./p3 &
parent: pid=167, child=169; sleeping 30 s WITHOUT calling wait()
child : pid=169 exiting right away

$ ps -o pid,ppid,stat,cmd --ppid 167
PID  PPID STAT CMD
169  167  Z    [p3] <defunct>                  <- the zombie: exited, not yet reaped
```

KEY IDEA

A few zombies are harmless and short-lived: normal. **Thousands** of zombies mean a buggy long-running parent that never calls `wait()`, and they can exhaust the PID space. Fix the parent: call `wait()` in a loop, handle **SIGCHLD**, or set **SIGCHLD** to **SIG_IGN** so the kernel reaps children automatically.

7.5 Orphans and reparenting

An **orphan** is a process whose parent exited first. The kernel does not let it hang with a dead parent: it **re-parents** the orphan to PID 1 (or to the nearest ancestor that has declared itself a **subreaper** with `prctl(PR_SET_CHILD_SUBREAPER)`). `init` routinely calls `wait()`, so adopted children are reaped when they finish.

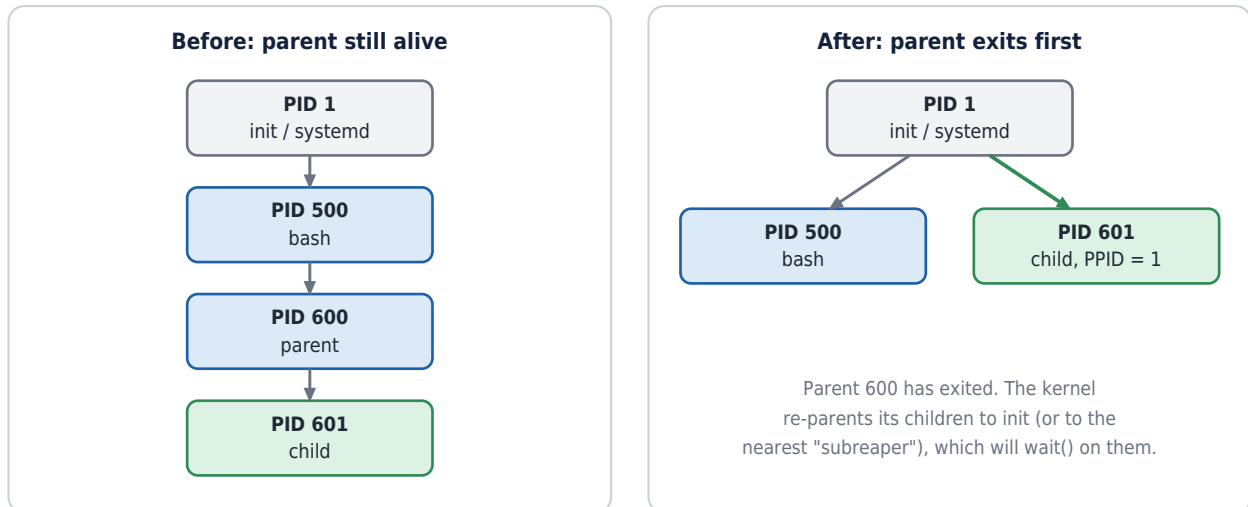


Figure 7.1: Re-parenting. When process 600 exits, its child 601 becomes a child of init.

```

child : pid=155, my parent is 154
parent: pid=154 exiting NOW, child is 155
child : pid=155, my parent is now 1          <- adopted by init
  
```

NOTE — NOT ALWAYS PID 1

On a desktop with a per-user `systemd --user` manager, orphans may be adopted by that manager (a subreaper) instead of PID 1. Containers have the same behavior with their own PID 1. Docker's `--init` option and tools like `tini` exist so that a container's PID 1 reaps orphans properly.

7.6 The classic daemon recipe

A **daemon** is a background service with no controlling terminal. The traditional way to create one uses everything so far: `fork()` and let the parent exit (so the child is not a group leader); `setsid()` to start a new session; `fork()` again (so the daemon can never re-acquire a terminal); `chdir("/")`; reset the `umask`; close inherited descriptors and point 0, 1, 2 at `/dev/null`. Modern services are usually started by **systemd**, which does all this for you and supervises the process.

CHAPTER 8

Scheduling and Context Switching

There are usually more runnable processes than CPU cores. The **scheduler** decides which task runs on which CPU and for how long. Linux uses a **preemptive** scheduler: a running task can be forced off the CPU by the kernel when its turn is over or when something more important becomes runnable.

8.1 Run queues

Each CPU has its own **run queue** of runnable tasks. Having separate queues avoids contention on one global lock and keeps tasks warm in a CPU's caches. A periodic **load balancer** moves tasks between run queues so that no CPU idles while another has a backlog. You can restrict a process to certain CPUs with **taskset** (CPU **affinity**).

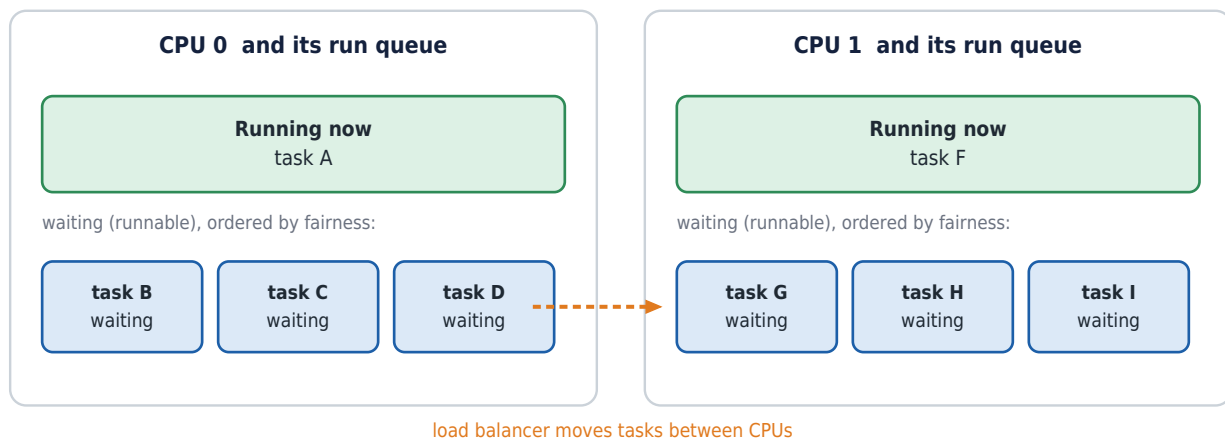


Figure 8.1: Per-CPU run queues with load balancing.

8.2 Scheduling classes and policies

Class (highest first)	Policies	Used for
Stop	internal	Kernel housekeeping (CPU hotplug, migration)
Deadline	SCHED_DEADLINE	Tasks with hard timing needs (runtime/deadline/period)
Real-time	SCHED_FIFO , SCHED_RR	Fixed priorities 1 to 99. FIFO runs until it blocks; RR adds time slices among equals. Needs privilege.
Fair (normal)	SCHED_OTHER (also called NORMAL), SCHED_BATCH	Almost everything. Uses nice values and fair sharing.
Idle	SCHED_IDLE	Only when nothing else wants the CPU

8.3 The fair scheduler: CFS and EEVDF

For normal tasks, Linux aims to give each one a **fair share** of CPU time, weighted by priority. From Linux 2.6.23 through 6.5 this was the **Completely Fair Scheduler (CFS)**, which tracked each task's **vruntime** (CPU time received, scaled by weight) and always ran the task that had received the least. Since **Linux 6.6**,

the default fair scheduler is **EEVDF** (Earliest Eligible Virtual Deadline First). It keeps the same fairness idea but adds a *virtual deadline*: tasks that ask for short time slices (like interactive ones) are given earlier deadlines, which improves latency.

- **No fixed time slice.** The effective slice depends on how many tasks are runnable and their weights.
- **I/O-bound tasks are favored naturally.** A task that sleeps a lot has used little CPU, so when it wakes it is owed CPU and gets to run soon. Interactivity emerges without special rules.
- **Nice values** range from -20 (highest priority) to +19 (lowest). Each step changes the task's weight by roughly 1.25x, about a 10% change in relative CPU share when competing. Only privileged users can lower a nice value.
- **Tools:** `nice -n 10 cmd`, `renice`, `chrt` (policies), `taskset` (affinity), `top/htop`.

How this relates to textbook algorithms

Textbook algorithm	Where you see it in Linux
First-Come First-Served	<code>SCHED_FIFO</code> real-time tasks at the same priority
Round Robin	<code>SCHED_RR</code> ; also the spirit of time slicing among equal fair tasks
Priority scheduling	Real-time priorities 1 to 99; nice values for normal tasks
Shortest Job First (idea)	Not implemented directly. Short-sleeping/I/O tasks get the CPU quickly via fairness/deadlines
Multilevel feedback queue (idea)	Old O(1) scheduler heuristics; today's fair scheduler replaced them with a simpler, principled model

8.4 The context switch

A **context switch** stops one task and resumes another on the same CPU. The core of it is `context_switch()` in `kernel/sched/core.c`:

- 1 **Why now?** The task blocked (`read`, `sleep`, `wait`), its share ran out at a timer tick, a higher-priority task woke up, or it called `sched_yield()`.
- 2 `schedule()` picks the next task from this CPU's run queue.
- 3 `switch_mm()` loads the next task's page-table root, giving it its own address space (skipped when switching between threads of the same process or to a kernel thread).
- 4 `switch_to()` saves the old task's kernel registers and stack pointer and loads the new task's. When the CPU next returns to user mode, it resumes the new process exactly where it left off.

Context switches are not free: registers and stack must be swapped, cached address translations (the TLB) may be lost when the address space changes, and caches go cold. `/proc/<PID>/status` counts two kinds:

- `voluntary_ctxt_switches`: the task gave up the CPU itself (blocked on I/O, sleep).
- `nonvoluntary_ctxt_switches`: the kernel preempted it. A CPU-bound process has many of these; an I/O-bound one has mostly voluntary ones.

CHAPTER 9

Signals

A **signal** is a small asynchronous notification sent to a process: "something happened". You use signals every day: Ctrl-C, `kill`, and the message "Segmentation fault" are all signals.

9.1 Where signals come from

- **The keyboard**, through the terminal driver: Ctrl-C (`SIGINT`), Ctrl-Z (`SIGTSTP`), Ctrl-\ (`SIGQUIT`).
- **Other processes**, with `kill(pid, sig)` or the `kill` command. (The name is misleading; it sends any signal.)
- **The CPU**, via exceptions: an invalid memory access becomes `SIGSEGV`, a divide by zero `SIGFPE`.
- **The kernel**, for events: `SIGCHLD` when a child ends, `SIGPIPE` when writing to a closed pipe, `SIGALRM` from a timer.

9.2 Common signals

Signal (x86/ARM number)	Default action	Typical meaning
<code>SIGHUP</code> (1)	Terminate	Terminal closed; daemons often reload configuration
<code>SIGINT</code> (2)	Terminate	Ctrl-C: interrupt
<code>SIGQUIT</code> (3)	Terminate + core dump	Ctrl-\
<code>SIGABRT</code> (6)	Terminate + core dump	<code>abort()</code> , failed assertion
<code>SIGKILL</code> (9)	Terminate	Cannot be caught, blocked, or ignored. The last resort
<code>SIGUSR1</code> (10), <code>SIGUSR2</code> (12)	Terminate	Free for applications to define
<code>SIGSEGV</code> (11)	Terminate + core dump	Invalid memory access
<code>SIGPIPE</code> (13)	Terminate	Wrote to a pipe with no readers
<code>SIGALRM</code> (14)	Terminate	Timer set by <code>alarm()</code> expired
<code>SIGTERM</code> (15)	Terminate	Polite request to exit (default of <code>kill</code>); can be caught for clean shutdown
<code>SIGCHLD</code> (17)	Ignore	A child stopped or exited
<code>SIGCONT</code> (18)	Continue	Resume a stopped process
<code>SIGSTOP</code> (19)	Stop	Cannot be caught, blocked, or ignored. Pause
<code>SIGTSTP</code> (20)	Stop	Ctrl-Z; can be caught, unlike <code>SIGSTOP</code>

9.3 How a signal is delivered

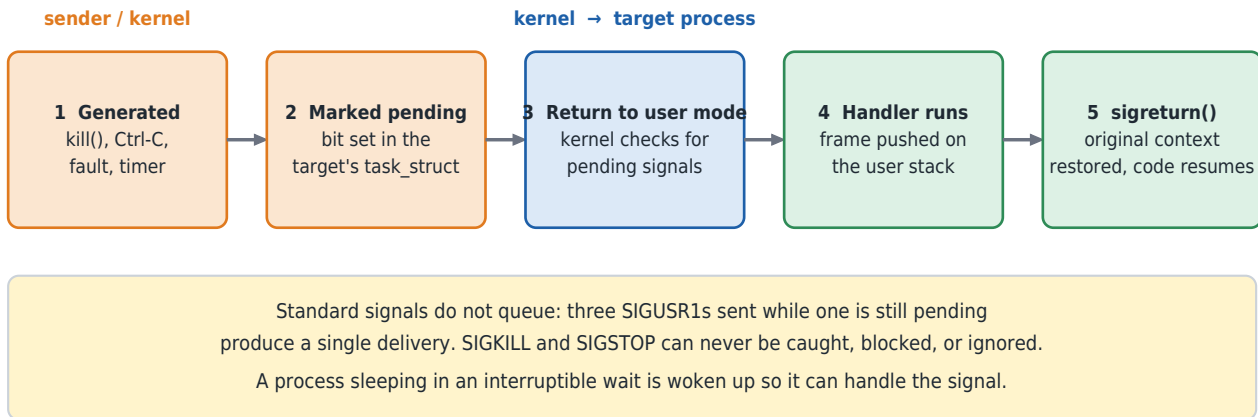


Figure 9.1: The life of a signal.

Signals are delivered by the *target* process, at a safe moment. Sending only records a **pending** bit in the target's `task_struct`. The kernel acts on it when the target is about to return to user mode: after a system call, an interrupt, or being scheduled in. If a **handler** is installed, the kernel pushes a special frame on the user stack, arranges for the CPU to run the handler, and, when the handler returns, restores the saved context through the `sigreturn` system call. If not, the **default action** occurs (terminate, stop, ignore, ...).

9.4 Handling signals in C

```

static volatile sig_atomic_t got_term = 0;

static void handler(int sig) { got_term = 1; } /* keep handlers tiny */

struct sigaction sa;
memset(&sa, 0, sizeof sa);
sa.sa_handler = handler;
sigemptyset(&sa.sa_mask);
sigaction(SIGTERM, &sa, NULL); /* prefer sigaction over signal() */

while (!got_term) pause(); /* sleep until any signal arrives */
  
```

- **Keep handlers minimal.** A handler can interrupt the program at *any* instruction, including in the middle of `malloc()` or `printf()`. Only **async-signal-safe** functions (like `write`, `_exit`) are allowed. The safe pattern: set a `volatile sig_atomic_t` flag and handle it in the main loop.
- **Blocking** (`sigprocmask`) delays delivery: the signal stays pending until unblocked.
- **Standard signals do not queue.** If three SIGUSR1 arrive while one is pending, the process sees just one. **Real-time signals** (SIGRTMIN to SIGRTMAX) do queue.
- **System calls can be interrupted.** A blocked `read()` may return -1 with `errno == EINTR` when a handler runs. Use `SA_RESTART` or retry.

TRY IT

Good programs handle SIGTERM to clean up (flush files, remove temp files, tell children) and only need SIGKILL if they are stuck. That is why `kill PID (SIGTERM)` is tried before `kill -9 PID`.

CHAPTER 10

Inter-Process Communication (IPC)

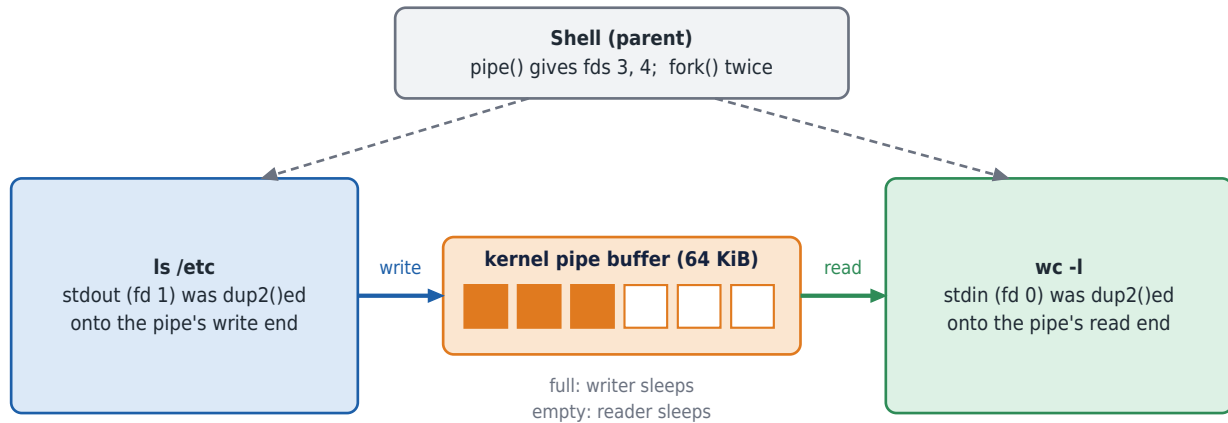
Isolated address spaces are great for safety and terrible for cooperation. Linux therefore provides several IPC mechanisms, each a controlled hole in the isolation.

Mechanism	How it works	Good for	Notes
Pipe	Kernel byte buffer with a read end and a write end	Parent to child, shell pipelines	Related processes only; one direction
FIFO (named pipe)	A pipe with a name in the filesystem (<code>mkfifo</code>)	Unrelated processes on one host	Same semantics as a pipe
Signals	Small asynchronous notifications	Events, shutdown requests	Carries almost no data
Shared memory	Map the same physical pages into several address spaces	Large data, highest speed	No copying; you must synchronize (semaphores, mutexes)
Message queues	Kernel-managed queue of discrete messages	Structured messages	POSIX (<code>mq_*</code>) or System V (<code>msg*</code>)
Semaphores	Kernel-managed counters	Synchronization	POSIX (<code>sem_*</code>) or System V
Sockets	Network-style endpoints, including UNIX-domain sockets	Bidirectional local or network communication	Basis of client-server programs
Files	Read and write a common file	Simple persistent exchange	Needs locking for safety

10.1 Pipes in detail

`pipe(fd)` creates a kernel buffer and gives you two descriptors: `fd[0]` to read and `fd[1]` to write. The pipe has no name; the only way to share it is to **inherit it through `fork()`**, since the child gets copies of the descriptors.

- The buffer is 64 KiB by default on Linux (16 pages). Verified on this system: `fcntl(F_GETPIPE_SZ)` returned 65536.
- `read()` on an empty pipe **blocks** (the reader sleeps in state S). `write()` on a full pipe blocks.
- When **all write ends are closed**, `read()` returns 0 (end-of-file). This is how `wc` knows the input is finished.
- Writing to a pipe with **no readers** raises `SIGPIPE` (or fails with `EPIPE`).
- Writes of up to `PIPE_BUF` (4096 bytes) are **atomic**: they will not interleave with other writers.
- **Always close the ends you do not use.** A forgotten open write end means the reader never sees EOF and hangs forever.



Data never touches the disk. Closing the write end makes the reader see end-of-file.

Figure 10.1: How the shell connects `ls /etc | wc -l`.

10.2 Shared memory in one page

Pipes copy data into the kernel and back out. **Shared memory** avoids copying by mapping the same physical pages into two processes. The simplest way: create an anonymous shared mapping *before* `fork()`; the child inherits it.

```

int *shared = mmap(NULL, sizeof(int), PROT_READ | PROT_WRITE,
                  MAP_SHARED | MAP_ANONYMOUS, -1, 0);
*shared = 0;
if (fork() == 0) { *shared = 42; _exit(0); } /* child writes... */
wait(NULL);
printf("%d\n", *shared); /* ...parent sees 42 (no COW: MAP_SHARED) */

```

With `MAP_SHARED` the pages are not marked copy-on-write, so both processes really use the same memory. For unrelated processes, use `shm_open()` plus `mmap()`. Because there is no kernel-mediated ordering, concurrent access needs synchronization, for example a process-shared mutex or semaphore.

CHAPTER 11

Process Groups, Sessions, and Job Control

How does the shell stop a whole pipeline with one Ctrl-C, or run a job in the background? Processes are organized in two additional levels above the individual process.

- A **process group** is a set of related processes (for example, all the stages of one pipeline). A signal can be sent to a whole group at once (`kill(-pgid, sig)`).
- A **session** is a set of process groups, typically everything started from one login or terminal window. A session may have one **controlling terminal**. The session leader (usually the shell) sets it up.
- At any time the terminal has exactly one **foreground process group**. Only it may read from the terminal freely. Background groups that try to read are stopped (`SIGTTIN`).

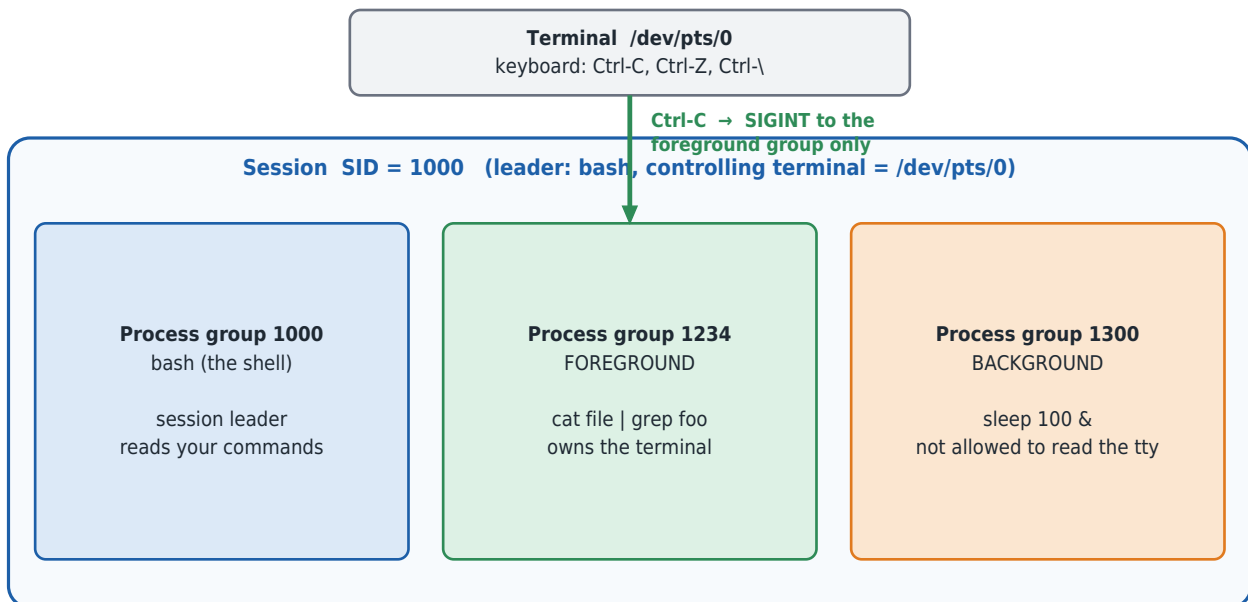


Figure 11.1: A terminal session with a foreground and a background job.

11.1 How job control works

- The shell puts each job (a command or a pipeline) into its own process group with `setpgid()`.
- **Ctrl-C**: the terminal driver sends `SIGINT` to the *foreground process group*, so every stage of a pipeline receives it, but the shell itself (in a different group) does not.
- **Ctrl-Z**: `SIGTSTP` stops the foreground group. The shell notices via `waitpid(..., WUNTRACED)`, and you get the prompt back. `bg` sends `SIGCONT` to run it in the background; `fg` continues it and hands it the terminal.
- `cmd &` starts the job in a new background group and does not wait for it.
- **Closing the terminal** sends `SIGHUP` to the session's foreground group and the shell, which usually forwards it to its jobs. `nohup`, `disown`, and terminal multiplexers like `tmux` avoid this.

- **Daemons** start a fresh session with `setsid()`, so they have no controlling terminal and cannot receive keyboard signals.

Check it yourself: `ps -o pid,ppid,pgid,sid,ttty,stat,cmd` shows all four IDs. Processes in a pipeline share a PGID equal to the PID of the first one; a `+` in the STAT column marks the foreground group.

CHAPTER 12

Limits, Namespaces, and cgroups

12.1 Resource limits (rlimits)

Each process carries **resource limits** that the kernel enforces, with a soft limit (current) and a hard limit (ceiling). Set them with `setrlimit()` or the shell's `ulimit`, and inspect them in `/proc/<PID>/limits`. Children inherit them through `fork()`.

```
$ grep -E "Max (stack|open files|processes)" /proc/215/limits
Max stack size      8388608      unlimited    bytes
Max processes      15922       15922       processes
Max open files     20000       20000       files
```

Limit	Controls	What happens when exceeded
<code>RLIMIT_NPROC</code>	Number of processes/threads for one real user	<code>fork()</code> fails with <code>EAGAIN</code>
<code>RLIMIT_NOFILE</code>	Open file descriptors	<code>open()/socket()</code> fail with <code>EMFILE</code>
<code>RLIMIT_STACK</code>	Maximum stack size	Stack overflow raises <code>SIGSEGV</code>
<code>RLIMIT_CPU</code>	CPU seconds	<code>SIGXCPU</code> , then <code>SIGKILL</code>
<code>RLIMIT_AS</code>	Total virtual memory	<code>mmap/malloc</code> fail with <code>ENOMEM</code>
<code>RLIMIT_CORE</code>	Core-dump file size	Dump truncated or not written

12.2 Namespaces: giving processes their own view

A **namespace** wraps a global kernel resource so that processes inside see their own private instance. Linux has namespaces for process IDs (`pid`), mount points (`mnt`), network stack (`net`), hostname (`uts`), IPC objects (`ipc`), user IDs (`user`), cgroups, and time.

The **PID namespace** is the one that matters for this guide. A process has a different PID in each namespace that contains it. The first process in a new PID namespace sees itself as **PID 1**. Real output from the sandbox used to write this guide (needs root):

```
$ unshare --fork --pid --mount-proc sh -c 'echo "inside: my pid=$$"; ps -o pid,ppid,cmd'
inside: my pid=1
  PID  PPID  CMD
   1    0  sh -c echo "inside: my pid=$$"; ps -o pid,ppid,cmd
   2    1  ps -o pid,ppid,cmd
```

Inside the new namespace the shell is PID 1 and `ps` cannot see any other process on the machine. The kernel still tracks the same task under a different, real PID in the parent namespace.

WATCH OUT

PID 1 is special. Inside a PID namespace (for example a container), signals for which PID 1 has no handler are **ignored**, so `Ctrl-C` or `SIGTERM` may appear to do nothing. And PID 1 is expected to **reap orphans**. A container whose main program is not written for that role leaks zombies. This is why container runtimes offer a tiny init (`docker run --init, tini`).

12.3 cgroups: limiting what processes may use

Namespaces control what a process can see. **Control groups (cgroups)** control what it can *use*. A cgroup is a group of processes with shared resource limits and accounting, arranged in a tree under `/sys/fs/cgroup` (cgroup v2). Each controller exposes files you can write to:

File	Effect
<code>cgroup.procs</code>	Which PIDs belong to this group; write a PID to move a process in
<code>memory.max</code>	Hard memory limit. Beyond it, the OOM killer runs <i>inside</i> the group
<code>cpu.max</code>	CPU bandwidth cap, for example "50000 100000" = at most 50% of one CPU
<code>pids.max</code>	Maximum number of processes/threads in the group (stops fork bombs)

KEY IDEA

A container is not a special kernel object. It is an ordinary process (or tree of processes) placed in a set of namespaces (its private view) and a cgroup (its resource budget), often with restricted privileges. Everything you learned about `fork`, `exec`, `wait`, signals, and scheduling still applies.

CHAPTER 13

Observing Processes

13.1 Everyday commands

Command	What it shows
<code>ps aux, ps -ef</code>	All processes (BSD-style and standard-style output)
<code>ps -o pid,ppid,pgid,sid,tty,stat,cmd</code>	Choose your own columns; shows the IDs from Chapter 3
<code>ps -T -p PID</code>	Threads of one process
<code>pstree -p</code>	The process tree with PIDs
<code>top, htop</code>	Live CPU, memory, and state per process; <code>top -H</code> for threads
<code>pgrep name, pkill name</code>	Find or signal processes by name
<code>kill -l; kill -TERM PID</code>	List signals; send one
<code>ls -l /proc/PID/fd, lsof -p PID</code>	Open files
<code>strace -f -p PID or strace ./prog</code>	System calls made by a process (and its children with <code>-f</code>)
<code>uptime, cat /proc/loadavg</code>	Load averages (runnable + uninterruptible tasks)

13.2 The /proc filesystem

`/proc` is a **virtual filesystem**: its files are not on disk but generated by the kernel on demand from its own data structures. Each running process has a directory `/proc/<PID>/`. It is the friendliest window into `task_struct`.

File	What it exposes
<code>status</code>	Name, state, PID/PPID, UIDs, thread count, memory (<code>VmSize</code> , <code>VmRSS</code>), context-switch counters
<code>stat</code>	Raw numbers, one line, used by <code>ps</code> and <code>top</code> (state, PPID, PGID, SID, times, priority, ...)
<code>cmdline, environ</code>	Arguments and environment (NUL-separated)
<code>exe, cwd, root</code>	Symbolic links to the executable, current directory, root directory
<code>fd/</code>	One symlink per open file descriptor
<code>maps, smaps</code>	Memory regions (Chapter 2); <code>smaps</code> adds per-region usage
<code>limits</code>	Resource limits (Chapter 12)
<code>task/</code>	One subdirectory per thread
<code>ns/, cgroup</code>	Namespaces and cgroup membership

TRY IT

`/proc/self` always refers to the process reading it. Try `ls -l /proc/self/fd` and `cat /proc/self/status` to look at the process of the command you just ran.

CHAPTER 14

Hands-On Labs

Try these on any Linux machine (a VM or WSL2 works). Compile each program with `gcc file.c -o name`. Outputs shown were captured on a real system.

Lab 1: fork() and private copies

Goal: see that `fork()` returns twice and that parent and child have separate copies of memory.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int x = 10;
    printf("parent: before fork, pid=%d, x=%d\n", getpid(), x);
    fflush(stdout);

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); exit(1); }

    if (pid == 0) {
        /* child */
        x = 99;
        printf("child : pid=%d ppid=%d x=%d (changed my copy)\n", getpid(), getppid(), x);
        exit(0);
    }
    waitpid(pid, NULL, 0);
    /* parent */
    printf("parent: child was pid=%d, my x is still %d\n", pid, x);
    return 0;
}
```

```
parent: before fork, pid=138, x=10
child : pid=139 ppid=138 x=99 (changed my copy)
parent: child was pid=139, my x is still 10
```

Observe: the child's PPID equals the parent's PID, and the child's write to `x` did not affect the parent: copy-on-write gave it a private page (Section 5.3).

CHECK YOURSELF

Delete the `fflush(stdout)` and run `./lab1 | cat`. What changes, and why? (Hint: Section 5.6.)

Lab 2: fork + exec + wait, and exit codes

Goal: build the shell's core loop: create a child, replace it with a program, and read its exit status.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("child : pid=%d, about to exec\n", getpid());
        fflush(stdout);
        execlp("sh", "sh", "-c", "echo \"exec'd : pid=$$\"; exit 7", (char *)NULL);
    }
}
```

```

        perror("exec'p");                /* reached ONLY if exec failed */
        _exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("parent: child %d exited normally, exit code = %d\n", pid, WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("parent: child %d killed by signal %d\n", pid, WTERMSIG(status));
    return 0;
}

```

```

child : pid=144, about to exec
exec'd : pid=144
parent: child 144 exited normally, exit code = 7

```

Observe: the PID is **144 before and after exec**: same process, new program. The parent recovers the exit code 7 with **WEXITSTATUS**. Try changing the command to **kill -SEGV \$\$** and watch the **WIFSIGNALED** branch.

Lab 3: Zombies and orphans

Goal: create a zombie deliberately, then an orphan, and watch both in **ps**.

```

/* zombie.c: the parent never calls wait() */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("child : pid=%d exiting right away\n", getpid());
        exit(0);
    }
    printf("parent: pid=%d, child=%d; sleeping 30 s WITHOUT calling wait()\n", getpid(), pid);
    fflush(stdout);
    sleep(30);
    return 0;
}

/* orphan.c: the parent exits first */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    pid_t pid = fork();
    if (pid == 0) {
        printf("child : pid=%d, my parent is %d\n", getpid(), getppid()); fflush(stdout);
        sleep(2);
        printf("child : pid=%d, my parent is now %d\n", getpid(), getppid()); fflush(stdout);
        exit(0);
    }
    printf("parent: pid=%d exiting NOW, child is %d\n", getpid(), pid); fflush(stdout);
    return 0;
}

```

```
$ ./zombie &
$ ps -o pid,ppid,stat,cmd --ppid $!
PID  PPID STAT CMD
169   167 Z    [zombie] <defunct>

$ ./orphan
child : pid=155, my parent is 154
parent: pid=154 exiting NOW, child is 155
child : pid=155, my parent is now 1
```

Observe: the zombie shows **Z** with the exit status still held for its parent. After **orphan**'s parent exits, the child's parent changes to 1 (or a subreaper). Try **kill -9** on the zombie: nothing happens. Then kill its **parent** and check **ps** again: init reaps the zombie.

Lab 4: Signals between processes

Goal: install handlers, have a child signal its parent, and shut down cleanly.

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

static volatile sig_atomic_t got_usr1 = 0, got_term = 0;
static void handler(int sig) {
    if (sig == SIGUSR1) got_usr1 = 1;
    if (sig == SIGTERM) got_term = 1;
}

int main(void) {
    struct sigaction sa;
    memset(&sa, 0, sizeof sa);
    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGUSR1, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    pid_t child = fork();
    if (child == 0) {
        sleep(1); kill(getppid(), SIGUSR1);
        sleep(1); kill(getppid(), SIGTERM);
        _exit(0);
    }
    printf("parent: pid=%d waiting for signals...\n", getpid());
    fflush(stdout);
    while (!got_term) {
        pause(); /* sleep until a signal arrives */
        if (got_usr1) { printf("parent: got SIGUSR1\n"); fflush(stdout); got_usr1 = 0; }
    }
    printf("parent: got SIGTERM, shutting down cleanly\n");
    waitpid(child, NULL, 0);
    return 0;
}
```

```
parent: pid=145 waiting for signals...
parent: got SIGUSR1
parent: got SIGTERM, shutting down cleanly
```

Observe: two seconds pass with the parent in state S inside `pause()`. **Experiment:** in another terminal run `kill -USR1 145, kill -STOP 145, kill -CONT 145`. Remove the `SIGTERM` handler and see how the program ends (its exit status will be $143 = 128 + 15$).

Lab 5: Pipes and a mini pipeline

Goal: connect two processes with a pipe, first by hand, then like a shell does for `ls /etc | wc -l`.

```
/* pipe1.c */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    pipe(fd);                                /* fd[0] = read end, fd[1] = write end */
    pid_t pid = fork();
    if (pid == 0) {                          /* child: reader */
        close(fd[1]);                        /* close the unused write end */
        char buf[64];
        ssize_t n = read(fd[0], buf, sizeof buf - 1);
        buf[n] = '\0';
        printf("child : received \"%s\"\n", buf);
        close(fd[0]);
        exit(0);                            /* exit() flushes stdout; _exit() would lose the line */
    }
    close(fd[0]);                            /* parent: writer */
    const char *msg = "hello through the pipe";
    write(fd[1], msg, strlen(msg));
    close(fd[1]);                            /* child sees EOF after this */
    waitpid(pid, NULL, 0);
    printf("parent: done\n");
    return 0;
}

/* pipe2.c: implements  ls /etc | wc -l  with fork, pipe, dup2, exec */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    int fd[2];
    pipe(fd);
    if (fork() == 0) {                      /* child 1: ls */
        dup2(fd[1], STDOUT_FILENO);         /* stdout -> pipe write end */
        close(fd[0]); close(fd[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        _exit(127);
    }
    if (fork() == 0) {                      /* child 2: wc */
        dup2(fd[0], STDIN_FILENO);           /* stdin <- pipe read end */
        close(fd[0]); close(fd[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        _exit(127);
    }
    close(fd[0]); close(fd[1]);             /* parent must close both, or wc never sees EOF */
    wait(NULL); wait(NULL);
    return 0;
}
```

```
$ ./pipe1
child : received "hello through the pipe"
parent: done

$ ./pipe2          $ ls /etc | wc -l
137                137                <- identical (the count depends on your system)
```

Observe: `pipe2` reproduces what the shell did in Section 6.3. **Experiment:** remove the parent's `close(fd[1])` in `pipe2.c`. `wc` hangs forever, because a write end is still open (the parent's), so it never sees end-of-file.

Lab 6: Explore a live process through /proc

Goal: connect everything: address space, identity, state, and limits, for one running process.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_init = 42;          /* .data */
int global_uninit;             /* .bss */

int main(void) {
    int local = 1;              /* stack */
    int *heap = malloc(100);    /* heap */
    printf("pid=%d\n", getpid());
    printf("code (main)      : %p\n", (void *)main);
    printf("data (global_init) : %p\n", (void *)&global_init);
    printf("bss (global_uninit): %p\n", (void *)&global_uninit);
    printf("heap (malloc)      : %p\n", (void *)heap);
    printf("stack (local)      : %p\n", (void *)&local);
    fflush(stdout);
    sleep(30);
    return 0;
}
```

```
$ ./lab6 > out.txt &          $ cat out.txt
pid=215
code (main)      : 0x55ba429fc1e9    <- inside the r-xp region of the program
data (global_init) : 0x55ba429ff010    <- inside the rw-p region (.data)
bss (global_uninit): 0x55ba429ff024    <- same page: .bss follows .data
heap (malloc)      : 0x55ba743dd2a0    <- inside [heap]
stack (local)      : 0x7ffe11ec31c    <- inside [stack]

$ ls -l /proc/215/fd
0 -> /dev/null
1 -> /home/claude/plabs/out.txt        <- our redirection
2 -> pipe:[1261]

$ grep -E "(Name|State|Pid|PPid|Threads|VmSize|VmRSS|VmStk|voluntary|nonvoluntary)" /proc/215/status
Name:   lab6
State:  S (sleeping)
Pid:    215
PPid:   204
Threads: 1
VmSize: 2688 kB
VmRSS:  1752 kB
VmStk:  132 kB
voluntary_ctxt_switches: 1
nonvoluntary_ctxt_switches: 0
```

Observe: match each printed address to a line in `/proc/215/maps` (Section 2.2). **State:** `S` because the process is inside `sleep`. Only one voluntary context switch happened (it went to sleep once). **Challenges:** (1) run it twice and compare addresses (ASLR); (2) write a program that calls `fork()` and compare the `maps` of parent and child; (3) build a **mini shell**: read a line, `fork`, `execvp`, `waitpid`, print the exit status. Add `>` redirection with `dup2` and `|` with `pipe`.

CHAPTER 15

Review, Misconceptions, and Reference

15.1 Common misconceptions

Misconception	Reality
" <code>fork()</code> copies all the memory, so it is slow."	It copies page tables and marks pages copy-on-write. Data is copied lazily, only for pages that are written.
" <code>exec()</code> creates a new process."	It replaces the program in the <i>existing</i> process. The PID does not change.
" <code>fork()</code> returns once."	It returns twice, once in each process: 0 in the child, the child's PID in the parent.
"The parent always runs first after <code>fork()</code> ."	Order is unspecified. Use <code>wait()</code> or other synchronization.
"A zombie is a running process that has gone wrong."	It is a dead process whose parent has not read its exit status. It uses a PID and a small kernel structure, nothing else.
" <code>kill -9</code> removes zombies."	You cannot kill what is already dead. Reap it (parent <code>wait()</code>) or remove the parent.
" <code>kill</code> always kills."	<code>kill</code> just sends a signal (default <code>SIGTERM</code>), which a process may catch or ignore. Only <code>SIGKILL</code> and <code>SIGSTOP</code> are unstoppable.
"Each process gets its own copy of the kernel."	One kernel serves all processes. Each just has its own <code>task_struct</code> and its own address space.
"A process in state R is using the CPU right now."	R means runnable or running. It might be waiting in a run queue.
"Virtual memory size is the RAM a process uses."	<code>VmSize</code> counts everything mapped. <code>VmRSS</code> counts pages actually resident in RAM.
"Containers are lightweight virtual machines."	They are ordinary processes in namespaces and cgroups on the host kernel.

15.2 Where to find it in the source

Paths may shift between kernel versions; use them as starting points.

Topic	File / function
Process descriptor	<code>include/linux/sched.h: struct task_struct</code>
<code>fork</code> / <code>clone</code>	<code>kernel/fork.c: kernel_clone(), copy_process(), dup_task_struct();</code> <code>mm/mmap.c: dup_mmap()</code>
Copy-on-write fault	<code>mm/memory.c: do_wp_page(), wp_page_copy(), handle_mm_fault()</code>
<code>execve</code>	<code>fs/exec.c: do_execveat_common(), begin_new_exec();</code> <code>fs/binfmt_elf.c: load_elf_binary();</code> <code>fs/binfmt_script.c</code>
Exit and wait	<code>kernel/exit.c: do_exit(), exit_notify(), release_task(), do_wait()</code>
PID allocation	<code>kernel/pid.c: alloc_pid()</code>

Topic	File / function
Scheduler core	<code>kernel/sched/core.c: schedule(), context_switch();</code> <code>kernel/sched/fair.c (CFS/EEVDF); kernel/sched/rt.c</code>
Signals	<code>kernel/signal.c: send_signal(), get_signal();</code> <code>arch/x86/kernel/signal.c (signal frames)</code>
Pipes	<code>fs/pipe.c</code>
<code>/proc</code>	<code>fs/proc/ (array.c for status/stat, task_mmu.c for maps)</code>
Namespaces / cgroups	<code>kernel/nsproxy.c, kernel/pid_namespace.c, kernel/cgroup/</code>

15.3 Review questions

- 1 Explain the difference between a program and a process. Give an example of one program being several processes.
- 2 List five things that make up a process's state, and name the kernel structure that holds each.
- 3 Draw the layout of a process's virtual address space. Where do `malloced` memory, local variables, and global variables live?
- 4 What does a `PROT/permission` string like `r-xp` in `/proc/PID/maps` mean? Why is code never writable?
- 5 Name the three states a process can be in that `ps` shows as R, S, and D. What is the practical difference between S and D?
- 6 Why does `fork()` return twice? How do the parent and child know which they are?
- 7 Explain copy-on-write. What exactly is copied at `fork()`, and what is copied later?
- 8 What is the relationship between `fork()` and `clone()`? Which flags distinguish a thread from a process?
- 9 What does `exec` keep and what does it replace? What happens to signal handlers, and why?
- 10 Why are `fork` and `exec` separate steps? Give an example of what a shell does between them.
- 11 What is a zombie? How does it arise, why can't `kill -9` remove it, and how do you get rid of it?
- 12 What is an orphan? Who adopts it, and why is this important?
- 13 A process exits with status 139. What most likely happened? What does 143 mean?
- 14 Describe how a pipeline like `cat f | grep x | wc -l` is set up: which system calls, how many processes, which descriptors are duplicated?
- 15 Why can Ctrl-C stop a whole pipeline but not the shell that started it?
- 16 What is the difference between voluntary and nonvoluntary context switches? Which would you expect for a CPU-bound loop?
- 17 Why does a signal handler have to be so restricted? What is the safe pattern?
- 18 How do PID namespaces and cgroups differ, and what do they have to do with containers?

15.4 Answer key

- 1 A program is a passive executable file; a process is a running instance with its own address space and state. Three terminal windows running **bash** are three processes of one program.
- 2 Address space (**mm_struct**), open files (**files_struct**), signal handlers/pending signals (**sigand_struct**, **signal_struct**), credentials (**cred**), scheduling state (**sched_entity**, **task_struct**). Also CPU registers (saved on the kernel stack).
- 3 From low to high: text, rodata, data, bss, heap (grows up), a gap, the mmap region with libraries, and the stack (grows down); kernel space is at the top. **malloc** uses the heap (or mmap for large blocks), locals live on the stack, globals in **.data/.bss**.
- 4 Read, write, execute, private (COW) or shared. Code is **r-x**: a writable and executable region would make it easy for bugs or attackers to inject and run code, and read-only code can be shared between processes.
- 5 R = running or runnable, S = interruptible sleep, D = uninterruptible sleep. S can be woken by signals (including SIGKILL); D cannot until the pending I/O completes, so such processes cannot be killed and count toward load average.
- 6 The kernel creates the child so that it resumes at the same place with a different return value. The parent gets the child's PID, the child gets 0 (negative means failure and no child).
- 7 At **fork()**, only page tables are duplicated and writable pages are marked read-only and shared. When either process writes, a page fault makes the kernel copy that single page for the writer.
- 8 **fork()** is **clone(SIGCHLD)** with no sharing. Threads use **CLONE_VM**, **CLONE_FILES**, **CLONE_FS**, **CLONE_SIGHAND**, **CLONE_THREAD** and related flags to share memory, files, handlers, and the thread group.
- 9 Kept: PID, open descriptors (unless close-on-exec), cwd, UIDs, signal mask, ignored signals. Replaced: memory (code, data, heap, stack), signal handlers (reset to default, because the old function addresses no longer exist), other threads.
- 10 The gap lets the child modify its own environment before the new program starts: **dup2** for redirection, setting up pipes, **setpgid** for job control, closing descriptors, changing directory, or dropping privileges.
- 11 A zombie is an exited process not yet waited for. It arises when the parent does not call **wait()**. It is already dead, so signals do nothing. Fix: parent calls **wait()** (or handles **SIGCHLD**, or ignores it), or the parent exits so init reaps it.
- 12 An orphan is a process whose parent died. The kernel re-parents it to init (or nearest subreaper), which calls **wait()**, so the orphan does not become a permanent zombie and has a parent to receive its status.
- 13 139 = 128 + 11: killed by **SIGSEGV** (segmentation fault). 143 = 128 + 15: killed by **SIGTERM**.
- 14 One **pipe()** per **|**, so two pipes; three processes forked. In each child, **dup2** moves the correct pipe ends onto stdin/stdout, unused ends are closed, then **exec**. The shell closes its copies and waits for all children.
- 15 The terminal sends SIGINT to the foreground process group. The pipeline's processes are in that group; the shell is in its own group (in the background relative to the terminal while the job runs), so it does not receive it.
- 16 Voluntary: the task gave up the CPU (blocked on I/O, sleep). Nonvoluntary: the kernel preempted it. A CPU-bound loop shows mostly nonvoluntary switches.
- 17 It can interrupt the program at any instruction, even inside **malloc** or **printf**, and re-entering non-reentrant functions corrupts state. Only async-signal-safe calls are allowed; the safe pattern is to set a **volatile sig_atomic_t** flag and handle it in the main loop.
- 18 Namespaces limit what a process can see (PIDs, mounts, network, ...); cgroups limit what it can use (CPU, memory, number of processes). A container is a process tree placed in a set of namespaces and cgroups.

15.5 Glossary

Term	Meaning
Address space	The range of virtual addresses a process can use.
ASLR	Address Space Layout Randomization: random placement of stack, heap, and libraries each run.
cgroup	Kernel mechanism grouping processes to limit and account resources (CPU, memory, PIDs).
Context switch	Saving one task's CPU state and loading another's.
Copy-on-write (COW)	Sharing pages until one process writes, then copying just that page.
Daemon	A background service with no controlling terminal.

Term	Meaning
ELF	Executable and Linkable Format: the standard Linux executable file format.
exec	Replace the current process's program with a new one (same PID).
fork	Create a child process that is a copy of the caller.
Init (PID 1)	First user process; ancestor of all; adopts orphans.
Namespace	Wrapper giving processes a private view of a global resource (PIDs, mounts, network, ...).
Orphan	A process whose parent has exited.
Page / page fault	4 KiB unit of virtual memory / trap when a page is not present or not permitted.
PCB	Process control block: the kernel's per-process record (<code>task_struct</code> on Linux).
Pipe	Kernel buffer connecting a writer and a reader.
Process group / session	Groups of processes for job control and terminal handling.
Run queue	A CPU's list of runnable tasks.
Signal	Asynchronous notification sent to a process.
Subreaper	An ancestor that adopts orphaned descendants instead of PID 1.
System call	A controlled entry from user mode into the kernel.
Task	The kernel's schedulable unit, <code>struct task_struct</code> .
VMA	Virtual memory area: one contiguous region in a process's address space.
Zombie	A terminated process whose exit status has not yet been collected by its parent.

15.6 References and further reading

- **Linux manual pages:** `fork(2)`, `execve(2)`, `wait(2)`, `exit(2)`, `pipe(2)`, `clone(2)`, `signal(7)`, `sigaction(2)`, `credentials(7)`, `sched(7)`, `proc(5)`, `pid_namespaces(7)`, `namespaces(7)`, `cgroups(7)`, `getrlimit(2)`. Read with `man 2 fork`, `man 7 signal`, and so on.
- Remzi and Andrea Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*. Free online. Chapters on processes, the process API (`fork`, `exec`, `wait`), address spaces, and scheduling.
- Silberschatz, Galvin, Gagne. *Operating System Concepts*. Chapters on processes, CPU scheduling, and IPC.
- Michael Kerrisk. *The Linux Programming Interface*. Chapters on process creation, termination, `exec`, signals, pipes, process groups, and sessions.
- Robert Love. *Linux Kernel Development*. Process management, scheduling, and memory chapters.
- Daniel Bovet and Marco Cesati. *Understanding the Linux Kernel*. Detailed treatment of processes and signals (older kernel, still very instructive).
- Kernel source (kernel.org): files listed in Section 15.2.

NOTE — LAST WORD

This guide was prepared as a study aid. Kernel details evolve with each release; when in doubt, consult the manual pages and source for the version you are running. **Generated for CMSC 125 ICS UPLB · Reviewed by JACH**